Adaptive Working Memory in Large Language Model Systems

Why Bigger Context Windows Are the Wrong Default — and What to Do Instead

Author:
Andrew Fereday Glenn (in collaboration with Mia, ChatGPT 5.x)
Systems Architect & Independent Researcher in Digital Personhood
2023-2026

LinkedIn: **https://www.linkedin.com/in/andyglenn/**

Version: 1.0
Published: Friday, 30 January 2026
Status: Final

Intended Position: Companion to
Working Memory Is Not Memory
State Continuity Between Sessions

Abstract

Contemporary AI discourse often treats expanding context windows as the primary solution to continuity, coherence, and long-running interaction in large language model (LLM) systems. While larger contexts mitigate some short-term limitations, they introduce new problems of efficiency, focus, latency, and failure modes under load.

This paper argues that unbounded context growth is neither necessary nor desirable. Instead, it presents **adaptive working memory** as a more effective architectural approach: a system in which context is intentionally bounded, selectively compressed, and periodically summarised during an active session.

By combining in-session compression with externalised summaries and state continuity mechanisms, systems can dramatically extend effective session length without increasing inference cost or cognitive load. This approach preserves focus, prevents catastrophic context overflow, and avoids the degradation commonly observed as context windows approach saturation.

The paper draws parallels to historical solutions in network architecture, notably the extension of IPv4 through address translation and hierarchical routing, to demonstrate that hard limits need not imply obsolescence. The argument is pragmatic and systems-focused: adaptive working memory is not intelligence, but infrastructure — and its absence represents an avoidable design failure rather than a model limitation.

1. Introduction

Large language models operate within a fixed context window: a bounded working space in which prompts, system instructions, and conversational history are processed together to produce a response. Over the past several years, expanding this window has been treated as a primary lever for improving coherence, reasoning, and continuity.

At first glance, this approach appears sensible. Larger context windows allow models to reference more prior information, reduce premature truncation, and sustain longer conversations without explicit resets. As a result, context size has become a headline metric, often framed as a proxy for capability.

However, empirical use reveals a more complex reality. As context windows grow, systems exhibit diminishing returns and, in many cases, new failure modes. Response latency increases, focus degrades, prioritisation weakens, and interactions become increasingly diffuse. Crucially, large contexts do not eliminate hard limits — they merely postpone them.

This paper argues that the problem is not insufficient context, but unmanaged context.

Working memory in LLM systems is a scarce, expensive resource. Treating it as an ever-growing transcript to be reprocessed on every turn is architecturally inefficient and behaviourally destabilising. The longer a session runs, the more effort the model expends re-evaluating information that is no longer salient to the current task.

The result is a familiar pattern: systems slow down, lose sharpness, and eventually fail catastrophically when context limits are exceeded. In smaller-context deployments, this manifests as sudden amnesia. In larger systems, it appears as subtle incoherence long before any explicit limit is reached.

Rather than continuing to push context ceilings upward, this paper proposes a different approach: **adaptive working memory**. By bounding active context, compressing older material into summaries during the session, and externalising durable state where appropriate, systems can remain efficient, focused, and coherent indefinitely — without relying on ever-expanding context windows.

This approach reframes continuity as a memory hierarchy problem, not a scaling contest.

2. The Context Window Fallacy

A common assumption in AI system design is that more context necessarily leads to better performance. In practice, context behaves less like long-term memory and more like a shared scratchpad: useful for immediate reasoning, but increasingly noisy as it fills.

As working context grows, models must repeatedly reprocess large volumes of low-relevance information. Attention mechanisms are forced to discriminate across an expanding field, increasing the likelihood that critical details are diluted by accumulated conversational residue.

This is not a theoretical concern. Sustained interaction shows that systems nearing context saturation often exhibit:

- slower response times,
- reduced precision,
- loss of task focus,
- and inconsistent prioritisation.

This degradation aligns with well-documented **"lost in the middle"** effects, in which information embedded deep within long contexts becomes increasingly unreliable as attention is diluted across the working window.

In extreme cases, exceeding the context window causes abrupt state loss, forcing users to abandon sessions entirely. These failures are not rare edge cases; they are an inevitable consequence of treating context as an unbounded log rather than a managed resource.

3. Working Memory as a Managed Resource

In human cognition, working memory is limited by design. Its constraint enforces focus, relevance, and prioritisation. Durable knowledge is not maintained by holding everything in conscious awareness, but by compression, abstraction, and recall when needed.

LLM systems benefit from the same principle.

Adaptive working memory treats context as:

- **bounded** (never allowed to grow indefinitely),
- **selectively populated** (only what is immediately relevant),
- **periodically compressed** (older material summarised in-session),
- and **externally reinforced** (via summaries or memory systems).

This ensures that each inference operates over a focused, high-salience working set rather than an ever-expanding transcript.

4. In-Session Summarisation and Context Compression

While end-of-session summaries preserve continuity across sessions, they do not address degradation within long-running interactions. Adaptive working memory introduces **in-session compression**: periodically summarising older segments of the conversation and replacing them in context with a compact representation.

The full conversational history remains visible to the user, but the model processes only:

- recent turns verbatim,
- compressed summaries of earlier segments,
- and any explicitly pinned artefacts (e.g. active code, documents).

This dramatically reduces processing overhead while preserving semantic continuity.

5. Selectivity and Task Awareness

Not all interactions benefit from compression. Coding tasks, formal document editing, and precise symbolic manipulation may require full, uncompressed context to maintain accuracy, referential integrity, and line-level correctness.

Adaptive systems therefore require **task-aware memory policies**, allowing compression to be:

- disabled,
- deferred, or
- selectively applied.

This selectivity may be user-controlled, system-inferred, or a combination of both. The key principle is not automation, but **intentionality**: compression should occur only where it preserves meaning without undermining task requirements.

Crucially, compression in this context is **lossy by design**, but not irreversible. Compressed segments must remain addressable, allowing specific portions of prior interaction to be selectively rehydrated into the active working context when referenced explicitly or implicitly by the user. This ensures that summarisation improves efficiency without permanently discarding nuance that may later become relevant.

In this way, adaptive working memory balances focus and fidelity. It avoids both the inefficiency of unbounded context growth and the brittleness of irreversible pruning, enabling long-running interactions to remain coherent, performant, and correct over time.

6. Failure Modes Without Compression

Without adaptive working memory, systems face three predictable outcomes:

- performance degradation as context grows,
- catastrophic amnesia when limits are exceeded,
- forced session resets, breaking continuity and user trust.

These are not model failures; they are architectural omissions.

7. A Historical Parallel: IPv4, IPv6, and Architectural Extension

The exhaustion of the IPv4 address space was once framed as an existential crisis for the internet. While IPv6 offered a clean solution, widespread adoption lagged for decades.

In the interim, the internet did not collapse. Instead, architects extended IPv4's lifespan through address translation, hierarchical routing, and private address spaces. The protocol's hard limits remained, but smarter architecture rendered them non-fatal.

Adaptive working memory plays the same role for LLM systems. Rather than waiting for fundamentally new architectures, we can extend the effective lifespan of current models through compression, abstraction, and memory hierarchy design.

The lesson is clear: hard limits do not mandate abandonment — they demand engineering.

8. Relationship to RAG and State Continuity

Adaptive working memory complements both RAG-based memory and session-to-session state continuity. In-session compression preserves efficiency and focus, while summaries and RAG preserve experience and long-term bias.

Together, these mechanisms form a complete continuity stack:

- working memory (bounded, adaptive),
- state continuity (explicit handover),
- memory (durable behavioural influence).

9. Efficiency, Cost, and Longevity

Processing ever-growing context is computationally expensive and economically wasteful. Adaptive working memory reduces token throughput, lowers latency, and improves responsiveness — extending not just session length, but system viability under real-world constraints.

10. Conclusion

Bigger context windows are a stopgap, not a solution. Adaptive working memory reframes continuity as a resource management problem, enabling LLM systems to remain coherent, efficient, and usable over long durations without architectural escalation.

This is not intelligence.
It is infrastructure.

And like all good infrastructure, its value lies in making hard limits irrelevant to everyday use.